

Linux Device Model

Wie der Driver Core Geräte, Treiber, Busse, Klassen und Lebensdauer in ein gemeinsames Modell einordnet

4.02

Das Device Model löst vier Treiberprobleme

01 Einheitliche Lebensdauer

Referenzen und release-Pfade folgen einem gemeinsamen Grundmodell.

02 Automatisches Binding

Busregeln ordnen passende Geräte und Treiber einander zu.

03 Explizite Hierarchie

Parent-Child-Beziehungen bilden Topologie und Abhängigkeiten ab.

04 Gemeinsame Sichtbarkeit

Geräte, Busse, Klassen und Treiber werden systemweit auffindbar.

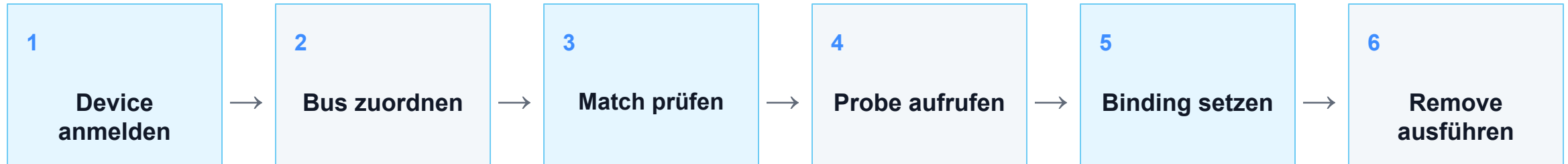
Ein Driver Core · viele Busse · konsistente Lebenszyklen · gemeinsame Beobachtbarkeit

Fünf Objekte teilen klar getrennte Rollen

Objekt	Aufgabe	Zentrale Beziehung
struct device	konkrete Geräteinstanz	parent, bus, driver, class
struct device_driver	Treiberidentität und Callbacks	bindet an passende Devices
struct bus_type	Matching und Busregeln	verwaltet Geräte und Treiber
struct class	funktionale Gerätesicht	gruppiert gleichartige Schnittstellen
struct kobject	Name, Hierarchie, Referenzen	Basis für sysfs-Einträge

Kein Objekt ersetzt die anderen: Das Modell entsteht aus ihren expliziten Beziehungen.

Binding ist ein geordneter Ablauf im Driver Core



Identität

Name, Bus, Firmwaredaten und Parent

Entscheidung

Bus-Match wählt eine Treiberkandidatur

Lebenszyklus

probe übernimmt, remove löst die Bindung

Vertrag

Erst erfolgreiches probe macht aus einer Kandidatur ein Binding.

Gerätehierarchie und funktionale Sicht sind verschieden

Perspektive	Beziehung	Beispiel
Physische Topologie	device.parent	Controller → untergeordnetes Gerät
Buszugehörigkeit	device.bus	PCI, USB, platform, I ² C
Treiberbindung	device.driver	genau ein gebundener Treiber
Funktionale Klasse	device.class	block, net, tty, input
Firmwarebezug	device.fwnode	Device Tree, ACPI oder Software Node

Parent beschreibt Besitz und Abhängigkeit; Class beschreibt die Funktion für andere Kernelbereiche.

struct device ist der gemeinsame Anker

```
struct device {  
    struct device *parent;  
    struct bus_type *bus;  
    struct device_driver *driver;  
    struct class *class;  
    struct kobject kobj;  
    void (*release)(struct device *);  
    struct fwnode_handle *fwnode;  
};
```

- | | | |
|-----------|---------------------|-------------------------------------|
| 01 | Identität | Name und konkrete Instanz |
| 02 | Topologie | Parent-Child-Beziehungen |
| 03 | Matching | Bus und gebundener Treiber |
| 04 | Lebensdauer | kobject, Referenzen und release |
| 05 | Sichtbarkeit | Klasse und spätere sysfs-Projektion |

Treiberbetreiber betten struct device häufig in eine größere, gerätespezifische Struktur ein.

Device, Bus und Driver steuern das Binding



Das Device bindet nicht selbst: Der Driver Core orchestriert die Regeln des jeweiligen Busses.

struct bus_type zentralisiert Busregeln

name	stabile Identität des Bustyps
match	prüft Device gegen Driver
probe	startet die treiberspezifische Initialisierung
remove	löst die Bindung kontrolliert
shutdown	ordnet Systemabschaltung des Geräts

Bus-Callbacks definieren Rahmen und Reihenfolge; der konkrete Driver implementiert die Gerätefunktion.

Ein Driver beschreibt Identität und Lebenszyklus

```
static struct platform_driver demo_driver = {  
    .probe = demo_probe,  
    .remove = demo_remove,  
    .driver = {  
        .name = "demo",  
        .of_match_table = demo_of_match,  
    },  
};  
  
module_platform_driver(demo_driver);
```

- | | | |
|----|--------------------|--|
| 01 | name | Treiberidentität im Driver Core |
| 02 | Match-Daten | Bus-ID oder Firmware-ID |
| 03 | probe | Ressourcen übernehmen und Zustand anlegen |
| 04 | remove | Binding lösen und externen Zugriff stoppen |
| 05 | Driver Data | gerätespezifischen Zustand am Device ablegen |

probe darf erst erfolgreich zurückkehren, wenn das Device vollständig nutzbar oder sauber deaktiviert ist.

Referenzzählung schützt Lebensdauer, nicht Zustand

Referenzoperation

`device_initialize ()`

`get_device ()`

`put_device ()`



kobject

kref zählt Besitzer
Name + Hierarchie



Objektzustand

Referenz > 0

Speicher bleibt erreichbar

Letzte Referenz

release gibt Speicher frei

Ein erreichbares Objekt kann bereits ungebunden oder inaktiv sein – Zustand separat synchronisieren.

Registrierung und Freigabe müssen symmetrisch sein

```
device_initialize(&obj->dev);  
obj->dev.parent = parent;  
obj->dev.bus = &demo_bus;  
obj->dev.release = demo_release;  
dev_set_name(&obj->dev, "demo0");  
  
ret = device_add(&obj->dev);  
  
/* Rückbau */  
device_del(&obj->dev);  
put_device(&obj->dev);
```

01 Initialisieren

Felder und release vor Veröffentlichung setzen.

02 Veröffentlichen

device_add macht die Instanz für Binding sichtbar.

03 Entfernen

device_del trennt; put_device gibt Referenz ab.

Jedes dynamisch angelegte Device braucht einen gültigen release-Callback für die endgültige Freigabe.

probe und remove markieren die Bindungsgrenzen

```
static int demo_probe(struct platform_device *pdev)
{
    struct demo *d;
    d = devm_kzalloc(&pdev->dev, sizeof(*d), GFP_KERNEL);
    if (!d)
        return -ENOMEM;

    platform_set_drvdata(pdev, d);
    return 0;
}

static void demo_remove(struct platform_device *pdev)
{
    /* externen Zugriff stoppen */
}
```

1

match

Bus erkennt passende Kandidatur

2

probe

Treiber übernimmt Ressourcen und Zustand

3

bound

Device und Driver sind verbunden

4

remove

Zugriff stoppen und Binding lösen

Fehlschlägt probe, muss jeder bis dahin aufgebaute Zustand konsistent zurückgerollt sein.

Managed Resources vereinfachen Fehlerpfade

API	Verwaltete Ressource	Wirkung
devm_kzalloc	Treiberzustand	Freigabe bei Detach
devm_ioremap_resource	MMIO-Mapping	Unmap automatisch
devm_request_irq	Interruptregistrierung	IRQ wird freigegeben
devm_add_action_or_reset	beliebige Cleanup-Aktion	sofortiger Reset bei Fehler
dev_set_drvdata	Zustand dem Device zuordnen	kein Ressourcenbesitz

devm verwaltet Lebensdauer, ersetzt aber nicht das Stoppen laufender Arbeit und externer Zugriffe.

Vier Regeln verhindern typische Lifecycle Fehler

01 release ist verpflichtend

Dynamische Devices brauchen einen endgültigen Freigabepfad.

02 put beendet Nutzung

Nach Abgabe der letzten Referenz darf kein Zugriff folgen.

03 drvdata braucht Synchronisation

Pointer allein schützt nicht gegen paralleles remove.

04 Rückbau bleibt symmetrisch

Arbeit stoppen, Schnittstellen sperren, Bindung lösen.

Referenzzählung, Binding und Laufzeitzustand sind drei verschiedene Dimensionen des Lebenszyklus.

Vier Sichten machen das Device Model beobachtbar

```
$ find /sys/devices -maxdepth 2 -type d

$ ls -l /sys/bus
$ ls -l /sys/class

$ readlink -f /sys/class/block/$DEV/device
$ readlink -f /sys/class/block/$DEV/device/driver
```

1

Topologie

Parent-Child-Pfad unter /sys/devices

2

Bus

Geräte, Treiber und Binding

3

Klasse

funktionale Sicht auf Geräte

4

Treiber

Symlink vom Device zum gebundenen Driver

Ausblick

sysfs, Uevents und udev im Zusammenspiel untersuchen